

Stage 13: Productization — Homework Sheet

All paths below are inside `homework/homework13/`.

Assignment Overview

This is the last homework of the course. It is deliberately self-contained: you do **not** need your project data, your project model, or anything from a previous week. The starter notebook generates its own dataset, so you can do this homework from an empty folder.

What you are practising is the one skill that is hard to rehearse on a real project — taking a trained model out of a notebook and putting it behind a working API that a *different* program can call.

In the lecture, we learned how to train a model, save it with `joblib`, load it back, write a Flask app that serves predictions, and call that app from a notebook with `requests`. **Now, you will build** the same thing end to end, by yourself, on data you generate.

What You Will Build

Four files, all inside `homework/homework13/`:

<code>homework13_productization_submission.ipynb</code>	the notebook you work in and submit
<code>model/model.pkl</code>	your trained model, saved with
<code>joblib</code>	
<code>app.py</code>	the Flask API
<code>README.md</code>	how to run the API and call both
<code>routes</code>	

Tasks

1. Train and save a model

- Generate a dataset with `make_regression(n_samples=100, n_features=2, noise=0.1, random_state=42)`. The starter does this for you.
- Fit a `LinearRegression` on it.
- Create the `model/` folder with `os.makedirs('model', exist_ok=True)` — **do this before you save**, or the save fails with `FileNotFoundError`.
- Save the model with `joblib.dump(model, 'model/model.pkl')`.
- Load it back with `joblib.load` and print one prediction, to prove the file is good.

Use `joblib`, not `pickle`. Both work; the lecture uses `joblib` and so should you.

2. Write `app.py`

`app.py` must **load the model once, when the app starts** — at the top of the file, next to the imports, not inside a route. A route that calls `joblib.load` on every request re-reads the file from disk every time a user asks for a prediction, which is the mistake this task exists to teach you to avoid.

It must serve **two routes**, both returning JSON, both using that one loaded model:

route	method	input	returns
/predict	POST	JSON body {"features": [f1, f2]}	{"prediction": <number>}
/predict/<f1>/<f2>	GET	two path parameters	{"prediction": <number>}

Two routes, one model, two different ways for a caller to reach it — a program posting JSON, and a person or a browser using a URL.

3. Handle bad input

Both routes must return a **JSON error and an HTTP 400**, not a traceback, when:

- the POST body is missing the **features** key, or has the wrong number of features
- a path parameter is not a number (e.g. /predict/abc/0.2)

4. Call your own API from the notebook

In your submission notebook, with the server running:

- `requests.post` to /predict with a JSON payload, and print the response
- `requests.get` to /predict/<f1>/<f2>, and print the response
- one deliberately bad call, and print the 400 and the error message

Leave that output visible in the notebook you submit. That output **is** your testing evidence.

5. Write README.md

Someone who has never seen your folder must be able to run this. It must contain:

- what the model does, in two sentences
- the exact command that starts the server (`python app.py`)
- **a copy-pasteable example of each route** — a `curl` command or a `requests` snippet, with the response it produces
- what happens on bad input

An API nobody can figure out how to call is not finished. The README is what makes it usable, which is the whole point of productization.

Step-by-Step

1. Copy `stage13_productization_homework-starter.ipynb` into `homework/homework13/`, rename it `homework13_productization_submission.ipynb`, and open it.
2. Run the data-generation and training cells. Confirm `model/model.pkl` exists.
3. Fill in the `app.py` cell and run it — it writes `app.py` to disk.
4. Run the launch cell. A separate terminal window opens and the server starts.
5. Run the `requests` cells. Fix `app.py` and restart the server until all three work.

6. Write `README.md`.
7. Save, commit, push.

Submission

- Save as `homework13_productization_submission.ipynb`, with output visible.
- Commit `app.py`, `model/model.pkl` and `README.md` alongside it.
- Due: next class session.

Grading Rubric (100 pts)

Task	Points
Model trained and saved to <code>model/model.pkl</code> with <code>joblib</code>	15
Model loaded once at startup , not per request	15
<code>POST /predict</code> returns a correct prediction from JSON	20
<code>GET /predict/<f1>/<f2></code> returns a correct prediction from path parameters	20
Error handling: JSON error and HTTP 400 on bad input	10
Notebook shows both routes and one bad call working	10
<code>README.md</code> with the start command and a runnable example of each route	10

Total: 100 points

Stretch Goals (Optional)

- Add a `POST /retrain` route that regenerates the data, refits the model, overwrites `model/model.pkl`, and reloads it into the running app **without restarting the server**.
- Add a `GET /plot` route returning a chart as a PNG.
- Add logging so every request is recorded with its input and its prediction.